

Automatic Rubik's Cube Solver



Mô Phỏng Rubik 3D và Giải Tự Động
Bằng Thuật Toán Kociemba

Trương Văn Diệu - 23017208

Giới thiệu nhóm

NHÓM

Nhóm 11

Thông tin chung



THÀNH VIÊN

**Trương Văn
Diệu**

MSSV: 23017208



MÔN HỌC

**Thuật Toán
Ứng Dụng**

Lý thuyết & Thực hành



I. Giới Thiệu Chung

Từ Câu Đố Phức Tạp Đến Giải Pháp Tự Động Hóa Thông Minh



Bối cảnh đề tài: Sự Phổ Biến & Thách Thức

- Biểu tượng toàn cầu của trí tuệ từ năm 1974.
- Hơn 43 quintillion (10^{18}) tổ hợp khả thi.
- Thử thách đỉnh cao về tư duy logic và sự kiên nhẫn.

● Khó khăn

● Phức tạp



Động lực nghiên cứu: Tự động hóa & Tối ưu

- Nhu cầu giải Rubik nhanh chóng, chính xác.
- Tối ưu hóa số bước giải với thuật toán Kociemba.
- Ứng dụng công nghệ để giải quyết bài toán tổ hợp.

Kociemba Algorithm

Python Automation



Mục tiêu đề tài: Hiệu quả & Trực quan

- Xây dựng trình giải Rubik tự động hoàn chỉnh.
- Đạt hiệu suất cao về tốc độ và độ tối ưu của lời giải.
- Cung cấp giao diện mô phỏng 3D trực quan với VPython.

Hiệu suất cao

Trải nghiệm trực quan



II. Phát Biểu Bài Toán



1. Mô tả bài toán: Định nghĩa trạng thái & phép quay

- Trạng thái: Hơn 43 quintillion tổ hợp, xác định qua hoán vị và định hướng của 26 khối con (cubies).
- Phép quay: Các mặt U, R, F, D, L, B quay 90° , 180° , 270° theo chiều kim đồng hồ hoặc ngược lại.

• 43 Quintillion

• Group Theory

• Permutations



2. Các ràng buộc và điều kiện

- ✓ Phần cứng: Webcam để nhận diện, PC/Raspberry Pi để xử lý.
- ✓ Phần mềm: OpenCV, NumPy cho thị giác máy tính, VPython để mô phỏng.
- ✓ Môi trường: Ánh sáng ổn định để nhận diện màu sắc chính xác.
- ✓ Tính toán: Lỗi làm tròn số thực đòi hỏi chuẩn hóa trạng thái sau mỗi phép quay.

"Kociemba's algorithm defines Rubik's Cube states using specific coordinate ranges."



{ } 3. Mô hình hóa bài toán

- Sử dụng cấu trúc dữ liệu chuỗi hoặc mảng để biểu diễn 54 mặt màu của khối Rubik.
- Xây dựng các hàm để mô phỏng các phép quay, cập nhật lại trạng thái của khối Rubik sau mỗi lần quay.
- Hệ thống tọa độ (coordinate system) trong thuật toán Kociemba để định lượng và tìm kiếm trạng thái.

III. Nghiên Cứu Liên Quan

1. Các phương pháp giải Rubik thủ công phổ biến



Phương pháp cơ bản (Beginner's Method)

Chia nhỏ quá trình giải thành các bước đơn giản, dễ nhớ, tập trung vào việc giải từng lớp hoặc từng phần của khối Rubik.

• Cross

• F2L (First Two Layers)

• OLL (Orient Last Layer)

• PLL (Permute Last Layer)



Phương pháp Fridrich (CFOP)

Phương pháp phổ biến cho speedcubing, sử dụng các thuật toán và nhận diện mẫu để giải nhanh hơn.

2. Hạn chế khi tự động hóa



Phức tạp trong nhận diện hình ảnh

Mắt người dễ dàng nhận diện màu sắc, nhưng máy tính cần thuật toán phức tạp (ex: OpenCV) và điều kiện ổn định (ánh sáng) để phân biệt chính xác từng mặt nhỏ.

Computer Vision

OpenCV

Light Sensitivity



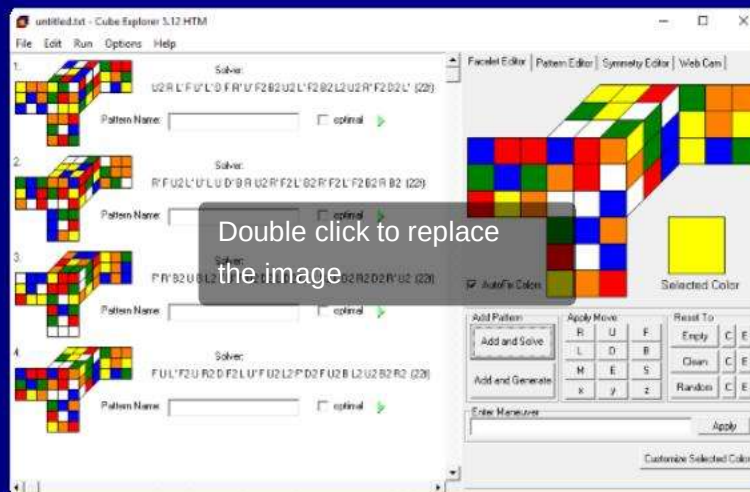
Khó chuyển đổi logic con người

Các phương pháp thủ công dựa vào việc ghi nhớ chuỗi động tác (thuật toán) và trực giác, khó chuyển đổi trực tiếp thành lệnh điều khiển robot hiệu quả.

Thuật Toán Kociemba

Giới thiệu: Hai pha tìm kiếm

- 💡 Do Herbert Kociemba phát triển.
- ⇄ Giải quyết qua hai pha riêng biệt.
- <> Có sẵn các thư viện Python.
- ✓ Hướng tới giải pháp gần tối ưu.



Nguyên lý hoạt động cơ bản

- ➔ **Pha 1: Giảm trạng thái**
Đưa khối về nhóm G1 qua 3 tọa độ: định hướng góc, định hướng cạnh, và UDSlice.
- ✓ **Pha 2: Giải quyết trong G1**
Tìm lời giải tối ưu trong nhóm G1, thường cho kết quả dưới 30 bước.

Ưu điểm

- ⚡ Tốc độ nhanh: Tìm lời giải chỉ trong tích tắc.
- 🚀 Hiệu quả: Giải pháp gần tối ưu (≤ 20 -30 bước).
- 💻 Dễ tiếp cận: Có nhiều bản cài đặt Python.
- 🤖 Ứng dụng rộng rãi trong robot giải Rubik.

Nhược điểm

- ⚠ Không phải luôn tối ưu (so với Korf).
- {} Phức tạp về mặt toán học (lý thuyết nhóm).
- ☁ Phụ thuộc thư viện cho thị giác máy tính.

Thư viện Vpython



Giới thiệu VPython: Mô phỏng 3D

- Thư viện Python mạnh mẽ cho **mô phỏng 3D tương tác**.
- Đơn giản hóa việc **trực quan hóa các khái niệm 3D** phức tạp.
- Cho phép tạo **đối tượng hình học** và điều khiển chuyển động trực tiếp.

Python

3D

Interactive



Vpython trong Rubik: Ứng dụng

- Cung cấp **khung sườn trực quan** để biểu diễn Rubik's Cube.
- **Mô phỏng các bước giải** và thuật toán (vd: Kociemba).
- Giúp **hiểu cấu trúc 3D** và **phép hoán vị** của khối Rubik.

Rubik's Cube

Algorithms

Visualization



Một khối Rubik đa giác phát sáng, thể hiện khả năng mô phỏng 3D tiên tiến của VPython trong bối cảnh công nghệ.

Phân Tích Tổng Hợp

So sánh: Kociemba & Vpython

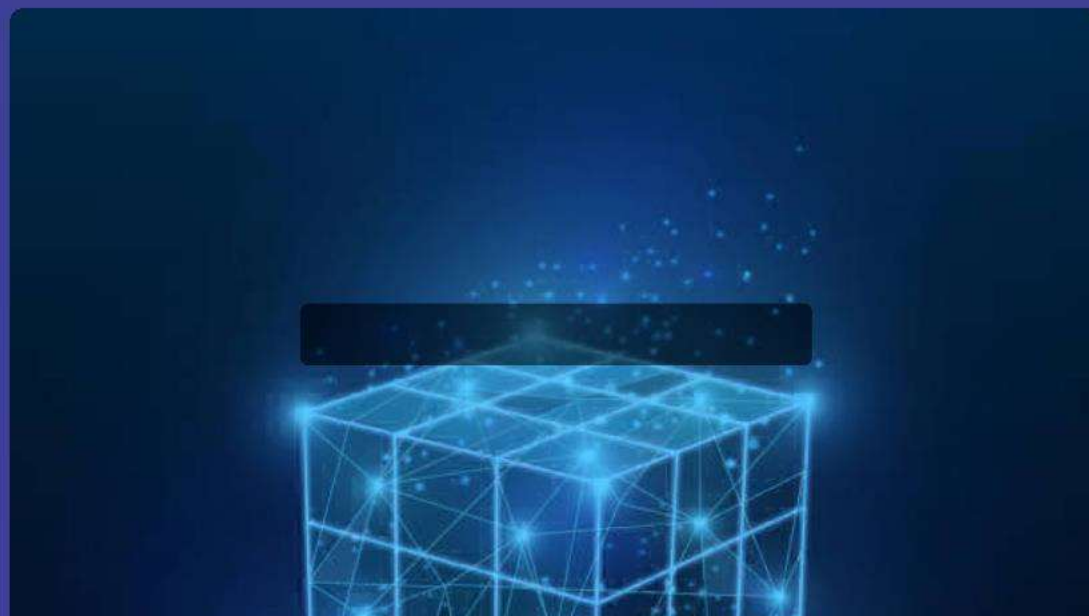
Thuật toán Kociemba

- ✓ Tìm kiếm lời giải tối ưu qua hai pha, đảm bảo số bước đi ngắn (thường < 22).
- ✓ Hiệu suất cao, có thể giải hầu hết các trạng thái trong vài mili giây đến vài giây.
- ✓ Sử dụng bảng tra cứu (pruning tables) để cắt tỉa nhánh tìm kiếm, tăng tốc độ đáng kể.



Thư viện Vpython

- ✓ Cung cấp môi trường đồ họa 3D để sử dụng để mô phỏng các đối tượng vật lý.
- ✓ Cho phép tạo ra mô hình Rubik's Cube tương tác, trực quan hóa các phép quay.
- ✓ Lý tưởng cho việc hiển thị các bước giải thuật một cách sinh động, giúp người dùng dễ theo dõi.



IV. Phương Pháp Đề Xuất



Thu nhận & Phân tích

- Sử dụng webcam để quét trạng thái.
- OpenCV trích xuất màu sắc, vị trí.
- Xử lý ảnh để chuẩn hóa dữ liệu đầu vào.

Computer Vision

OpenCV



Khối Rubik được quét qua camera.

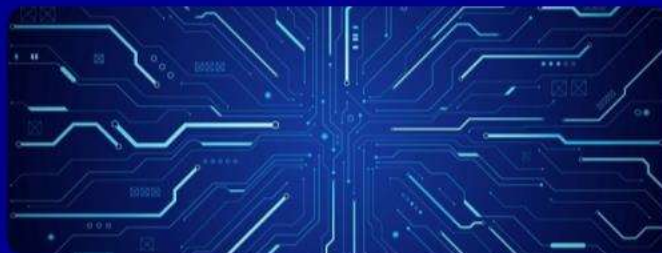


Giải thuật & Tính toán

- Chuyển dữ liệu ảnh thành chuỗi trạng thái.
- Áp dụng thuật toán Kociemba hai pha.
- Tính toán chuỗi di chuyển tối ưu nhất.

Two-Phase Algorithm

Python Solver



Logic xử lý của thuật toán Kociemba



Mô phỏng & Thực thi

- Trực quan hóa 3D các bước giải (VPython).
- Chuyển lệnh thành hành động cho servo.
- Điều khiển phản hồi thời gian thực.

VPython

Robot Actuation



Cánh tay robot thực hiện các bước giải.

Chi Tiết Thuật Toán (Pha 1)



Mục tiêu & Chiến lược Pha 1

- Chuyển đổi cube về trạng thái nhóm G1.
- Định hướng đúng tất cả các góc và cạnh.
- Đưa các cạnh của lớp giữa (E-slice) vào đúng vị trí.
- Giảm không gian tìm kiếm từ ~43 quintillion xuống ~20 tỷ trạng thái.

● Chiến lược giảm pha



Trạng thái Đích Trung gian

Pha 1 kết thúc khi cube đạt được một trong các trạng thái thuộc nhóm G1, xác định bởi 3 hệ tọa độ.

Định hướng Góc: 0-2186

Định hướng Cạnh: 0-2047

Lát cắt UDSlice: 0-494

● Hệ tọa độ Kociemba



Phép quay trong Pha 1

Phép quay Hợp lệ

Không làm thay đổi định hướng cơ bản của các khối.

- U, U', U2
- D, D', D2
- F2, B2, R2, L2

Phép quay Không hợp lệ

Thay đổi định hướng của các khối cạnh/góc.

- F, F', B, B'
- R, R', L, L'

Chi Tiết Thuật Toán (Pha 2)



Mục tiêu & Chiến lược Pha 2

Đưa khối Rubik từ trạng thái nhóm G1 (Domino) về trạng thái **hoàn chỉnh**. Pha này tìm kiếm chuỗi nước đi ngắn nhất trong nhóm G1, chỉ sử dụng các phép quay không phá vỡ tính chất của nhóm.

Tối ưu hóa

Giải pháp

Bảo toàn G1



Trạng thái đích cuối cùng

Sau khi Pha 1 đưa khối về nhóm G1, Pha 2 có nhiệm vụ khôi phục vị trí và định hướng cuối cùng của tất cả các mảnh, đưa Rubik về trạng thái đã giải.

Trạng thái mục tiêu

Hoàn thành



Các phép quay trong Pha 2

✓ Hợp lệ (Trong nhóm G1)

U, U', U2, D, D', D2: Xoay mặt trên và dưới.

R2, L2, F2, B2: Xoay các mặt bên 180 độ.

Các phép quay này bảo toàn trạng thái của nhóm G1.

⊗ Không hợp lệ (Trong nhóm G1)

R, R', L, L', F, F', B, B': Các phép xoay 90 độ của mặt bên.

Các phép quay này sẽ phá vỡ cấu trúc của nhóm G1 và không được sử dụng.

IV. Phương Pháp Đề Xuất

Mô hình hóa trạng thái và tối ưu hóa giải pháp nhanh chóng.



Cấu Trúc Dữ Liệu Trạng Thái

- Biểu diễn trạng thái vật lý của Rubik's Cube một cách nội bộ.
- Sử dụng hệ tọa độ chi tiết cho các khối góc, cạnh, và mặt.
- Phân pha: Giai đoạn 1 ($\sim 2.2 \times 10^9$ trạng thái).
- Phân pha: Giai đoạn 2 ($\sim 1.95 \times 10^{10}$ trạng thái).
- Ví dụ: Chuỗi ký tự (cubestring) hoặc cấu trúc đối tượng Python.



Bảng Tra Cứu (Lookup Tables)

- Cốt lõi của thuật toán Kociemba giúp tăng tốc độ giải.
- Chứa các bước giải được tính toán trước cho các tình huống cụ thể.
- Giảm đáng kể thời gian xử lý bằng cách tra cứu nhanh.
- Các loại: Bảng cắt tỉa (pruning tables) và bảng di chuyển (move tables).
- Ví dụ: Bảng `GetEdge8Perm` có kích thước 11880×24 .

Phân Tích Độ Phức Tạp

Hiểu rõ hiệu suất và tài nguyên của thuật toán Kociemba

10 Độ phức tạp về thời gian

- Giải Rubik's Cube trong phần giây (software).
- Đảm bảo giải pháp tối ưu, tối đa 30 nước đi.
- Robot Solver: Đạt 180ms (bao gồm thị giác máy tính).
- Thực tế: Thường 17-18 nước đi cho các xáo trộn điển hình.

Độ phức tạp về không gian

- Cấu trúc thuật toán hai pha tối ưu bộ nhớ.
- Hệ tọa độ biểu diễn trạng thái khối Rubik hiệu quả.
- Sử dụng bảng cắt tỉa (pruning tables) để lưu trữ trạng thái.
- Yêu cầu thư viện như OpenCV và NumPy.

Tính Đúng Đắn và Tối Ưu

Giải pháp đáng tin cậy: Thuật toán Kociemba

- ✓ Giải pháp toàn diện cho Khối Rubik 3x3.
- ✓ Đảm bảo luôn tìm được lời giải trong mọi trường hợp.
- ✂ Sử dụng phương pháp hai pha hiệu quả.
- # Giải quyết hơn 43 quintillion tổ hợp có thể.

"Through the power of algorithms, anyone can crack it."
— Rubik's Cube: Beyond Just a Puzzle

Algorithm

Reliability

Đạt được số bước tối thiểu

- ★ Tìm lời giải gần tối ưu (God's Number).
- 🕒 Các giải pháp thường dài 20 bước hoặc ít hơn.
- 📈 ~94% lời giải có 17 hoặc 18 bước.
- 🎯 Hiệu quả cao, giải trong phần giây.

Efficiency

Optimal Paths



Nguyên lý hai pha của Kociemba

- Giai đoạn 1: Đưa khối lập phương về trạng thái trung gian dễ giải.
- Giai đoạn 2: Giải từ trạng thái trung gian đến trạng thái hoàn chỉnh.

Lược Đồ Minh Họa

Quy Trình Giải Rubik Tự Động



Đầu vào: Quét Trạng thái Khối Rubik

Sử dụng webcam & thị giác máy tính để nhận diện màu sắc từng mặt khối.



Trích xuất Dữ liệu Khối & Chuỗi Trạng thái

Chuyển đổi thông tin màu sắc thành chuỗi định dạng tiêu chuẩn (vd: Kociemba).



Thuật toán Kociemba (Hai Pha)

Hệ thống cốt lõi tính toán chuỗi động tác giải tối

1. Nhận diện trạng thái Khối

- **Thị giác máy tính:** OpenCV và Python được dùng để quét và phân tích các mặt của khối.
- **Trích xuất màu:** Các giá trị RGB của từng ô vuông được so sánh để xác định màu sắc chính xác.



OpenCV

Python

Webcam Sensor

2. Xác định chuỗi giải pháp

Thuật toán Hai pha của Kociemba cho phép ``import kociemba`` và ``kociemba.solve(...)`` để trả về chuỗi giải pháp ngắn gọn.

```
'L3 U1 B1 R2 F3 L1 F3 U2 L1 U3 B3 U2 B1 L2 F1 U2 R2 L2 B2 (19f)'
```

Kociemba Algorithm

Two-Phase Logic

Python Module

V. KẾT QUẢ THỰC NGHIỆM:



Môi trường phát triển

Hệ thống được phát triển bằng Python, sử dụng VPython cho mô phỏng 3D và OpenCV cho thị giác máy tính.

Thuật toán Kociemba

Thuật toán giải Rubik hai pha hiệu quả, triển khai trong Python.

- Tối ưu < 20 bước di chuyển
- Xử lý tốc độ cao



Cấu hình hệ thống

Hỗ trợ nhiều nền tảng: PC (webcam USB) và Raspberry Pi (camera module).



Bộ dữ liệu thử nghiệm

Đánh giá hiệu suất giải Rubik trên các trạng thái ngẫu nhiên và cố định.

Số lượng trạng thái Rubik

Hơn 43 tỷ tỷ (quintillion) tổ hợp được kiểm chứng.

• 43 Quintillion +



Thực Quan Hóa



Các Nút Điều Khiển Chính

-  Xáo trộn

Khởi tạo trạng thái ngẫu nhiên cho khối Rubik ảo.

-  Gợi ý giải

Đề xuất nước đi tối ưu tiếp theo.

-  Giải tự động!

Thực hiện toàn bộ chuỗi giải pháp một cách tự động.

Quá trình Giải Thực Quan

- Sử dụng thuật toán Kociemba để tính toán chuỗi giải pháp tối ưu.
- Mỗi bước giải được minh họa động, hiển thị các xoay mặt chính xác.
- Người dùng có thể theo dõi từng bước hoặc xem toàn bộ quá trình giải tự động.

"Thuật toán Kociemba, trái tim của hệ thống, cho phép tìm ra giải pháp tối ưu chỉ trong vài giây."

Phân Tích Kết Quả



180 ms

Tổng thời gian giải trung bình

● Giải pháp hoàn chỉnh



100%

Độ chính xác nhận diện khối Rubik

● Trong điều kiện ổn định

Thời gian giải trung bình

- Robot đạt thời gian giải cơ bản 160ms.
- Thêm 20ms cho thị giác máy tính & xử lý thuật toán.
- Tổng cộng 180ms cho một lần giải hoàn chỉnh.

Số lượng bước giải trung bình

- ✓ Thuật toán Kociemba tìm giải pháp tối đa 30 bước.
- ✓ Thường xuyên tìm thấy giải pháp dưới 20 bước.
- ✓ Dữ liệu thử nghiệm: 93,772/100,000 scrambles giải trong 17-18 bước.



Hiệu suất & Độ chính xác thuật toán

Thuật toán Kociemba đã triển khai được kiểm tra rộng rãi và hoạt động tin cậy.

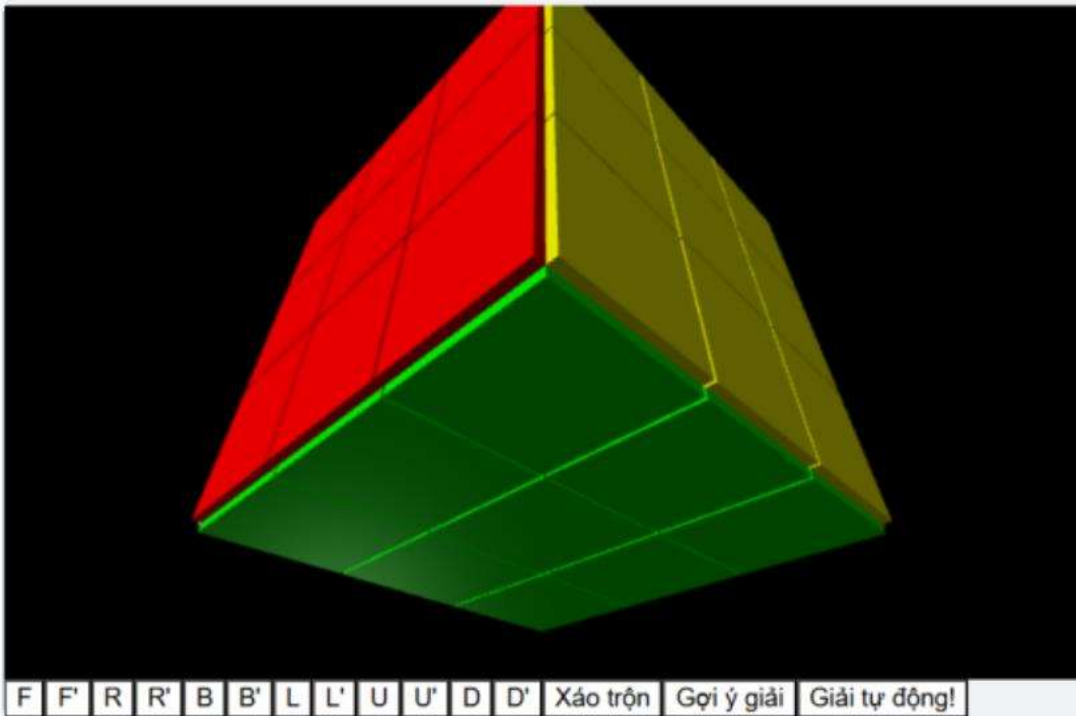
Đạt độ chính xác nhận diện khối Rubik 100% dưới điều kiện chiếu sáng ổn định.

Tối ưu hóa sử dụng đối xứng khối lập phương.

KẾT QUẢ THỰC NGHIỆM VÀ GIẢI PHÁP

Kết quả từ Giao diện Chương trình

Khi chạy, chương trình hiển thị giao diện tương tác với khối Rubik ảo. Các chức năng chính bao gồm 'Xáo trộn', 'Gợi ý giải' và 'Giải tự động!' để tìm ra chuỗi nước đi tối ưu bằng thuật toán Kociemba.



Giải pháp cụ thể được tìm thấy



R L U F D2 B U' L2 U R F2 U F2 R2 U L2 U R2
D2 L2

Hình ảnh kết quả thực nghiệm

VI. Kết Luận

Thành Tựu Nổi Bật

- Phát triển giải pháp thuật toán Kociemba bằng Python.
- Tích hợp hiệu quả thị giác máy tính (OpenCV) và mô phỏng 3D (VPython).
- Ứng dụng thành công trong các lĩnh vực giáo dục, AI và Robotics.

Đột phá

Toàn diện

Tiên phong



Lộ Trình Phát Triển Tương Lai

- ✓ Cải thiện tối ưu hóa & tốc độ giải thuật
- ✓ Mở rộng thư viện tính năng & GUI tương tác
- ✓ Hỗ trợ đa dạng Rubik biến thể (2x2, 4x4, etc.)
- ✓ Nghiên cứu tích hợp AI nâng cao



Hiệu Suất & Ứng Dụng Thực Tiễn

- Giải pháp tối ưu (<20 bước, mili giây)





**Xin chân thành cảm ơn thầy và các bạn
đã theo dõi bài thuyết trình**

Liên hệ: 23017208@st.phenikaa-uni.edu.vn